

Group Project Proposal

Improving Repository Navigation and Bug Localization in Minimal Software Engineering Agents

Group ID: 25

DSA4213: Natural Language Processing for Data Science
Semester 1, AY2026/2027

Administrative Information

Group members	ZHANG YUHANG(A0351628B) WANG CHI(A0357310M) CHEN WENWEN(A0351705J) JIANG ZHANGZHANG(A0355183B) CHEN XINGYU(A0355370E)
Project direction	Software Engineering Agents – Repository Navigation and Bug Localization.
Repository	https://github.com/235559649/DSA4213_group_work

Project Summary

Software engineering agents such as mini-SWE-agent can autonomously explore repositories, modify code, and execute tests, but repository exploration is largely driven by free-form shell interaction. This project studies whether structured repository navigation and failure-aware bug localization can improve the efficiency and reliability of software repair agents. We will extend mini-SWE-agent with a Python-oriented repository indexer that ranks relevant files and functions based on issue descriptions, code structure, and runtime failure evidence. We will compare the proposed agent with the original mini-SWE-agent on real bug-fixing tasks, evaluating repair success rate, localization accuracy, number of explored files, shell commands, token usage, runtime, and failure patterns.

1. Problem, Research Question, and Motivation

Software engineering agents must identify relevant code before producing a useful patch. In a large repository, an issue description may not name the responsible files or functions, and relevant logic may be distributed across several locations. For example, an incorrect discount calculation may require examining input handling, pricing, and rounding functions before identifying the cause.

This project targets bug-fixing assistance for developers and maintainers of Python repositories. We investigate whether structured repository information and runtime failure evidence can help mini-SWE-agent^[7] locate relevant code more efficiently. An agent-based approach is appropriate because debugging is iterative: code inspection and execution provide evidence that informs subsequent actions.

We will retain the existing repair loop and ordinary execution feedback, while adding ranked localization context and updates based on usable failure evidence. The aim is to assess the additional value of this guidance, rather than assume that more structure necessarily improves repair.

1.1 Primary Research Question

RQ1. Can structured repository navigation and runtime failure evidence help mini-SWE-agent localize relevant code more efficiently without reducing bug-fixing success?

1.2 Secondary Research Questions

RQ2. Does improved localization reduce exploration costs, including file inspections, model steps, token usage, and runtime, after accounting for localization overhead?

RQ3. What additional value do repository structure and runtime failure evidence provide beyond textual issue-to-code relevance?

1.3 Definition of Success

Success would mean improved localization accompanied by better repair performance, or comparable repair performance with lower exploration costs. Repair success will be determined by the independent evaluation described in Section 5; improved localization alone will not establish successful repair.

Pilot experiments on development tasks will inform quantitative success criteria, including the targeted improvement in localization or exploration efficiency and the acceptable change in repair performance. These criteria will be fixed before held-out evaluation. Comparisons and trajectory analysis will identify benefits, trade-offs, and failure cases. Where evidence is insufficient to establish comparable performance, we will report that uncertainty.

2. Related Work and Proposed Contribution

2.1 mini-SWE-agent

Software engineering agents alternate between model decisions, repository operations, and execution feedback. SWE-agent shows that the agent-computer interface (ACI) affects repository-level software repair through the design of commands and feedback^[12]. Building on this insight, we adopt **mini-SWE-agent** as our baseline: a separate, minimal implementation combining a language model, a command-execution environment, and a linear interaction history^[7]. Its shell interface supports the core operations needed for repair—repository search, code inspection, editing, and test execution. By default, the v2 implementation issues these operations through native bash tool calls, while legacy configurations instead parse commands from model-generated text^[8].

The teal path in Figure 1 shows the agent loop: *decide, act, observe, and update history*. Given the issue and interaction history, the model selects an action, which the environment executes as a shell command. The resulting output, errors, and test results are formatted as observations and appended to the history, alongside the model’s response, to inform the next decision^[7]. Across iterations, the agent can locate relevant code, investigate or reproduce a bug, edit files, and run

accessible tests. These are available actions rather than a fixed sequence: new evidence may prompt further inspection or another edit. The loop continues until submission or another stopping condition, such as a resource limit. The repository diff provides a candidate patch whose correctness is assessed separately.

Our extension preserves the model adapter, execution environment, and repair loop. It supplies ranked localization context before model decisions and updates that context when execution yields usable failure evidence. The baseline retains ordinary execution feedback, including failures, so the comparison isolates the added value of explicit candidate reranking.

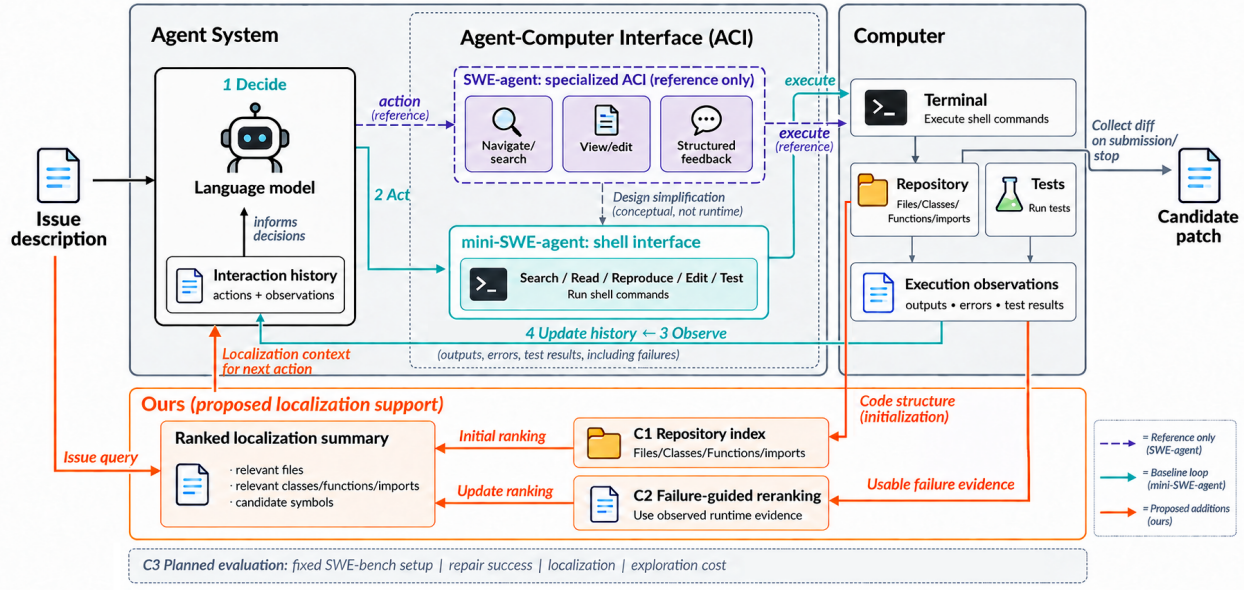


Figure 1: Overview of mini-SWE-agent and the proposed localization extension.

2.2 SWE-bench

SWE-bench evaluates repository-level software repair using real GitHub issues and executable tests^[4]. We propose to use **SWE-bench Verified**, a human-screened subset of 500 tasks from Python repositories^[9]. Its screening helps reduce ambiguity in issue descriptions and problems with evaluation tests. Its Python scope also matches our repository indexer, making it suitable for examining whether improved localization translates into successful repair.

From this task pool, we initially plan to select approximately 20 tasks for development and 100 additional tasks for evaluation, covering multiple repositories. The development tasks will support debugging and refinement of the localization components. The evaluation tasks will be reserved for comparing the baseline and proposed variants after the method is fixed. We may adjust these target sizes based on runtime and API costs measured during development, before fixing the evaluation task list. The two sets will remain disjoint, and all compared systems will use the same evaluation tasks.

2.3 Repository Localization Methods

Repository localization uses issue descriptions, code structure, and execution evidence to identify relevant files and functions. These information sources support complementary stages of the search: textual relevance provides initial candidates, structural relationships guide further inspection, and runtime observations help revise the search as repair proceeds, as shown in Table 1.

Table 1: Prior methods relevant to this study.

Category	Approach	Relevant mechanism	Role in this study
Agent framework	mini-SWE-agent ^[7]	Shell actions and execution feedback in a minimal agent loop.	Provides the baseline repair loop.
Textual retrieval	BugLocator ^[14]	File ranking using textual similarity and historical bug information.	Motivates the lexical localization comparison.
Staged localization	Agentless ^[11]	Hierarchical localization followed by repair and validation.	Separates candidate selection from patch correctness.
Structural navigation and reasoning	LocAgent ^[2]	Navigation over code entities and dependency relationships.	Motivates structural support for code search.
	CoSIL ^[3]	Iterative code-graph search with context pruning.	Motivates refinement of localization context.
	RepoNav ^[1]	File-centered organization of snippets and candidate functions.	Motivates structured presentation of candidates.
	GraphLocator ^[6]	Graph-guided reasoning about symptoms and edit locations.	Distinguishes observed symptoms from repair targets.
Execution-informed localization	AutoCodeRover ^[13]	Structural search with optional spectrum-based fault localization.	Connects code structure with test evidence.
	IssueExec ^[5]	Test retrieval and execution-trace analysis.	Links execution evidence to candidate edit locations.
Proposed integration	This study (planned)	Repository indexing and reranking from observed failures.	Updates localization context within the mini-SWE-agent loop.

Textual retrieval and staged localization. BugLocator ranks source files using report-code textual similarity and historical bug information^[14]. Agentless organizes issue resolution into localization, repair, and validation, progressively narrowing the code considered for modification^[11]. Together, these approaches provide a basis for selecting candidate locations before generating a patch. We therefore include a lexical localization condition to examine how much guidance issue-code matching provides within mini-SWE-agent. Assessing localization alongside repair will show whether improved candidate selection translates into successful fixes.

Structural navigation and evidence organization. LocAgent represents code entities and dependencies in a heterogeneous graph, allowing the agent to navigate relationships between relevant locations^[2]. CoSIL combines iterative code-graph search with context pruning to refine the information considered during localization^[3]. RepoNav organizes retrieved snippets around files

and candidate functions, supporting comparison between possible targets within a file^[1]. These studies show how code relationships and candidate organization can guide repository exploration beyond textual matching. They motivate our Python repository index, which records files, classes, functions, and imports, and supplies a ranked localization summary for subsequent agent decisions.

Execution-informed localization. AutoCodeRover combines structure-aware code search with optional spectrum-based fault localization^[13]. IssueExec retrieves relevant tests and analyzes their execution traces to identify candidate edit locations^[5]. These methods establish execution as a source of evidence linking an issue to the code involved in its behavior. GraphLocator further motivates examining the relationship between observed symptoms and locations requiring modification^[6]. Building on this rationale, our proposed component maps tracebacks, failing-test identifiers, and error messages observed during repair to indexed code entities. It then updates the candidate rankings and supplies the revised summary to the agent before its next decision.

Positioning of this study. The proposed system combines initial localization from repository structure with ranking updates from the agent’s own execution trajectory. Its focus is the connection between observed failures, indexed code entities, and the localization context used in subsequent decisions. The original mini-SWE-agent provides the repair loop and ordinary execution feedback; our extension adds an explicit ranking update mechanism. This design supports a controlled study of how structured localization context affects repair success and exploration cost. Table 1 summarizes the relevant mechanisms and the design choices they motivate.

2.4 Our Contribution

Prior work already supports structural navigation and execution-informed localization. Our contribution is a focused extension and controlled study within mini-SWE-agent, asking whether these signals improve localization efficiency without reducing repair success. We retain the three contributions defined in the group proposal:

- **C1: Repository indexing.** A Python-oriented index of files, classes, and functions that supports localization from the issue description.
- **C2: Failure-aware refinement.** A mechanism that maps observed runtime failures to code entities and updates suspicious-location rankings.
- **C3: Empirical evaluation.** A comparison with the original mini-SWE-agent of repair performance and repository exploration efficiency.

3. Agent and Task Setup

This section defines the common task environment used by the original mini-SWE-agent baseline and our localization-augmented variants. The software repair setting is kept unchanged across experimental conditions, while only the localization information supplied to the agent is varied. This controlled setup allows us to study whether structured repository information and runtime failure evidence improve repository navigation, bug localization, and repair efficiency.

3.1 Task

Each task consists of a natural-language software issue together with the corresponding version of a Python repository. The agent must inspect the repository, identify source code relevant to the reported problem, modify the code, and produce a patch that resolves the issue without introducing regressions.

The task follows the repository-level software repair setting used by SWE-bench^[4]. For each task, the agent is given:

- the natural-language issue description;
- the corresponding repository checkout;
- access to the repository through the execution environment; and
- test or execution feedback generated through the agent’s own actions.

The agent is not given the reference patch, benchmark evaluation labels, or ground-truth localization information. These are reserved exclusively for offline evaluation.

The primary output of each run is a final repository patch. In addition, we record the observable agent trajectory, including model responses, selected actions, shell commands, inspected files, localization rankings, code edits, test outputs, token usage, and runtime information. These trajectories will support the localization and exploration-efficiency analyses described in Section 5.

3.2 Operating Environment

The agent will operate in an isolated execution environment compatible with the SWE-bench evaluation infrastructure. Each task environment contains the corresponding repository version together with its required dependencies and available testing infrastructure.

Within the environment, the agent can perform normal software engineering operations, including:

- navigating the repository using shell commands;
- reading and searching source files;
- executing Python programs and project-specific commands;
- modifying source files;
- inspecting Git diffs; and
- running available tests or reproduction scripts.

The official SWE-bench evaluation harness will be used only for the final assessment of the submitted patch. Tests executed by the agent during its trajectory provide intermediate feedback but do not replace the benchmark’s official evaluation.

For reproducibility, we plan to use the task-specific containerized environments provided by the SWE-bench infrastructure. Different repositories may require different Python and dependency

versions, so each task will retain its benchmark-specified environment rather than being forced into a single Python version.

All experimental conditions will use the same repository versions, execution environments, and resource constraints. The final SWE-bench release, container configuration, and mini-SWE-agent version will be pinned before held-out evaluation.

3.3 Agent and Experimental Conditions

mini-SWE-agent provides the common software repair agent and interaction loop for all experimental conditions^[7]. At each step, the language model receives the issue description and current interaction history, selects an action, executes that action in the repository environment, observes the result, and uses the updated history to decide its next action.

Our system does not replace this repair loop. Instead, it augments the context available to mini-SWE-agent with explicit localization information. This design keeps the underlying repair agent fixed while allowing us to study the contribution of different localization signals.

The study will consider four main experimental conditions:

- A. Original mini-SWE-agent.** The agent explores the repository using its standard shell-based workflow without additional localization guidance.
- B. Lexical localization.** The agent receives an initial ranking of repository locations based on textual relevance between the issue description and indexed source-code entities.
- C. Structure-aware localization.** The agent receives localization context that combines lexical relevance with structural information extracted from the Python repository index.
- D. Structure-aware localization with failure-aware refinement.** The agent begins with the localization information used in Condition C and receives an updated ranking when usable run-time failure evidence becomes available.

A single fixed language model will be used across Conditions A–D so that performance differences cannot be attributed to different model capabilities. The same mini-SWE-agent version, base prompt, execution environment, resource limits, and generation settings will be used in all conditions.

We will access the language model through OpenRouter using the group API key and initial USD 15 credit provided by the course. Our provisional model choice is `openai/gpt-5.4-mini`, subject to development-pilot checks of availability, compatibility, performance, and cost. The final model identifier, supported reasoning and generation settings, and resource limits will be recorded and fixed before held-out evaluation and used consistently across Conditions A–D.

3.4 Localization Information and Agent Tools

The original mini-SWE-agent retains its normal shell-based interaction with the repository. Our extension adds a localization layer that provides structured guidance rather than restricting the agent to a predefined set of files.

Before the repair trajectory begins, the repository indexer analyzes the Python source tree and records code entities including:

- repository-relative file paths;
- classes;
- functions and methods;
- import relationships; and
- selected textual and structural metadata used for localization.

Given the issue description, the localization module generates a ranked list of candidate files and code entities. For the initial implementation, we plan to provide the agent with the top five ranked files. For each file, up to two highly ranked functions or methods will be included when function-level candidates are available.

Each localization entry may contain:

- the repository-relative file path;
- the class, function, or method name;
- the candidate type;
- its relevance score or rank; and
- a short description of the evidence supporting the ranking.

For example, the localization context supplied to the agent may have the following form:

1. `src/parser.py`
`function: parse_header`
`score: 0.87`
`evidence: strong issue-text match + structural relevance`
2. `src/io/reader.py`
`function: read_table`
`score: 0.71`
`evidence: related import + lexical relevance`

The agent remains free to inspect other repository locations using its standard shell tools. Therefore, localization acts as navigation guidance rather than as a hard search constraint.

Lexical Localization

For Condition B, lexical localization will use a BM25-style retrieval method. Each indexed code entity will be represented using information such as its file path, identifier, class or function name, docstring, and selected source text. The issue description will serve as the retrieval query.

For file-level fusion, each file’s lexical score will be the maximum BM25 score among its indexed entities. Files with positive scores will form the lexical ranking L , with ties broken by repository-relative path.

The resulting lexical ranking provides a simple retrieval baseline for measuring whether explicit issue-to-code matching alone improves mini-SWE-agent’s repository exploration.

Structure-Aware Localization

For Condition C, lexical evidence will be supplemented with repository structure. The initial structural representation will include containment relationships between files, classes, and functions, together with import relationships between Python modules.

To generate a structural file ranking, we will use a one-hop import-neighbor heuristic. Let Q contain up to the first ten files with positive scores in the lexical ranking L . For each seed file q , let $N(q)$ contain the repository files that it imports or that import it, using statically resolvable internal imports and excluding the file itself. Each seed distributes a weight of $1/r_L(q)$ equally among its distinct neighbors. A file’s structural score is the sum of the weights it receives:

$$s_{\text{struct}}(f) = \sum_{q \in Q} \frac{\mathbb{I}[f \in N(q)]}{r_L(q) \max(1, |N(q)|)}.$$

Files with positive structural scores form ranking S , with ties broken by lexical rank and then repository-relative path. Condition C will fuse L and S using RRF. If no structural candidates are available, it will retain L . Containment relationships will organize class and function hints within files; the proposed structural fusion will operate at the file level. Parameters will be fixed after the development pilot and before held-out evaluation.

Because lexical and structural signals may produce scores on different numerical scales, we plan to combine their rankings using reciprocal rank fusion (RRF):

$$\text{RRF}(x) = \sum_{r \in R} \frac{1}{k + r(x)},$$

where $r(x)$ is the position of candidate x in one input ranking and R is the set of rankings being combined. We provisionally set $k = 60$, which will remain fixed unless development-set experiments reveal a clear implementation problem.

A candidate absent from an input ranking contributes zero for that ranking; tied fused scores are ordered by lexical rank and then repository-relative path.

This approach allows different evidence sources to contribute to the final candidate ordering without requiring their raw scores to be directly comparable.

Failure-Aware Localization

For Condition D, runtime failures provide an additional ranking signal. When a command, reproduction attempt, or test produces a usable failure, the system extracts available evidence such as:

- exception type;
- traceback frames;
- repository files appearing in the traceback;
- class, function, or method names appearing in the traceback;
- failing-test identifiers; and

- relevant identifiers mentioned in error messages.

These observations are mapped back to entities in the repository index. Candidates appearing directly in repository traceback frames receive the strongest runtime evidence, while related test and error-message information provides secondary evidence.

A runtime-based ranking is then fused with the existing lexical and structural rankings using the same RRF mechanism. This produces an updated candidate list without discarding evidence derived from the original issue description.

The revised localization summary is supplied to mini-SWE-agent before a subsequent repair decision.

The reference patch and benchmark evaluation labels are never used to construct or update these rankings.

3.5 Benchmark and Task Selection

The study will use tasks from **SWE-bench Verified**, a human-screened subset of repository-level software engineering problems^[9]. Its Python repository setting is compatible with our Python-oriented repository index and provides executable tests for determining whether a generated patch resolves the corresponding issue.

We provisionally plan to use approximately 20 tasks as a development set and up to 100 additional tasks for held-out evaluation.

The development tasks will be used to:

- implement and debug the repository indexer;
- validate lexical and structural localization;
- develop the failure-aware refinement mechanism;
- select practical prompt and interface formats;
- estimate token usage, runtime, and API cost; and
- verify that the planned resource limits are feasible.

The development and held-out sets will remain disjoint. Once the pilot stage is completed, the held-out task list, model configuration, resource limits, and comparison protocol will be frozen before the evaluation results are inspected.

Tasks will be selected across multiple repositories where possible so that conclusions are not dominated by a single codebase. The same task instances will be used across all four experimental conditions.

For file-level localization evaluation, only tasks whose reference patch contains at least one modified Python source file will contribute to the localization metrics. Tasks that cannot be scored for localization may still contribute to the overall repair evaluation when applicable.

The exact task identifiers and repository distribution will be published in the project repository together with the final experimental configuration to support reproducibility.

3.6 Stopping Conditions and Failure Recovery

Each agent run will operate under a fixed resource budget. For the initial proposal, we provisionally plan the following limits:

- maximum of 50 agent interaction steps;
- maximum wall-clock runtime of 30 minutes per task; and
- maximum total model-token budget of approximately 150,000 tokens per run.

These limits are intended to prevent unusually difficult tasks from consuming disproportionate resources. Pilot experiments on the development set may be used to reduce or adjust these limits if necessary. The final limits will be frozen before held-out evaluation and applied equally to all experimental conditions.

A run terminates when one of the following conditions is reached:

- the agent submits a final patch;
- the maximum number of interaction steps is reached;
- the token or API-cost budget is exhausted;
- the wall-clock execution limit is exceeded; or
- an unrecoverable environment failure prevents further execution.

A patch is not considered successful merely because tests executed by the agent during repair pass. Final repair success is determined separately using the official SWE-bench evaluation harness, as described in Section 5.1.

Failed commands and failed tests are treated as observations rather than immediate termination conditions. In Conditions A–C, this feedback remains part of the normal mini-SWE-agent interaction history. In Condition D, usable failures additionally trigger the localization refinement mechanism.

A runtime failure is considered usable for localization if at least one piece of evidence can be mapped back to the repository index. This occurs when:

- a traceback contains a repository source-file path;
- a traceback contains an indexed class, function, or method;
- a failing test produces a traceback that reaches project source code; or
- an error message explicitly mentions an indexed source entity.

If none of these conditions is satisfied, the failure does not trigger reranking and the agent continues using its existing localization context.

The failure-aware repair process can therefore be summarized as:

Initial ranking → Agent exploration → Execution → Failure evidence → Updated ranking →
Continued repair → Final patch.

This design ensures that runtime evidence augments the original issue-based localization rather than replacing it, while allowing the baseline and proposed systems to operate under the same repair environment and resource constraints.

4. Proposed System or Study Design

The proposed system extends mini-SWE-agent with structured repository navigation and bug localization while keeping its existing repair capability. The workflow consists of repository indexing, initial localization, the mini-SWE-agent repair process, and, when runtime failure evidence is available, failure-aware refinement.

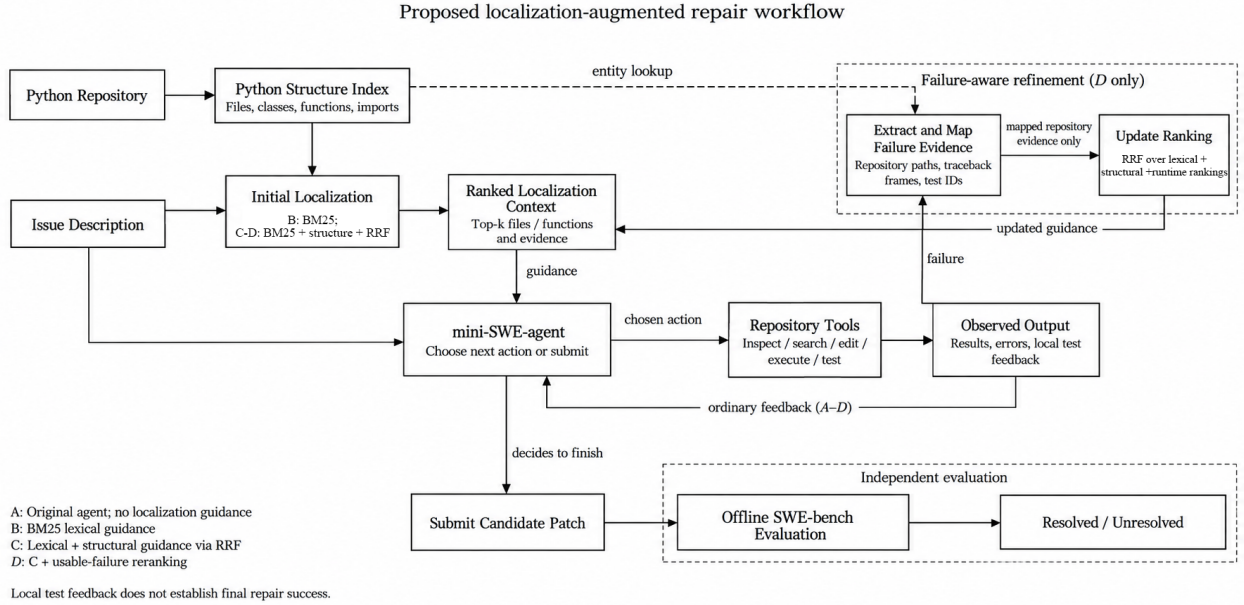


Figure 2: Proposed system architecture. Candidate submission and official SWE-bench evaluation are separate; intermediate test feedback does not establish final repair success.

4.1 Baseline

The baseline is the original mini-SWE-agent using its standard repository exploration and repair loop. Given a software issue, the agent explores the repository through shell commands, inspects relevant files, modifies source code, and runs tests to validate its patch. The baseline and proposed methods will use the same experimental settings, with localization information being the main difference between conditions.

4.2 Repository Indexing

Before repair begins, the repository indexer analyzes the Python repository and extracts structural information at the file, class, and function levels. The index records file paths, classes, functions and methods, and import relationships. Python AST information will be used where appropriate to extract these structures consistently.

The resulting index provides a compact representation of the repository for subsequent localization. Its exact internal representation will be finalized during implementation.

4.3 Initial Localization

Given an issue description, initial localization follows Section 3.4: Condition B uses BM25-style lexical ranking, while Conditions C and D combine lexical and structural file rankings using RRF.

The resulting ranked list of suspicious locations is provided to mini-SWE-agent as additional context, allowing the agent to inspect likely relevant code earlier while retaining access to the full repository. Pilot work may refine implementation parameters and the presentation format, which will be fixed before held-out evaluation.

4.4 Failure-Aware Refinement

When a command, reproduction attempt, or test produces a failure whose observed output can be mapped to an indexed repository entity, the system extracts the usable runtime evidence described in Section 3.4.

This evidence is used to update the previous ranking of suspicious code locations before the next repair attempt. Locations implicated by the failure can therefore receive greater attention in the updated ranking. This allows localization to adapt during debugging instead of relying only on the original issue description and static repository structure.

4.5 Repair Loop

The localization results are passed to mini-SWE-agent as additional context. The agent then follows its existing repair workflow to inspect code, make modifications, and execute tests. The agent submits a candidate patch when it decides to finish. Passing tests during the trajectory provides feedback but does not establish repair success, which is determined independently by the official SWE-bench harness as described in Section 5.1.

For the failure-aware condition, a failed attempt with usable runtime failure evidence triggers an updated localization ranking, which can guide the next repair attempt. The loop continues until a patch is submitted or the predefined execution budget is exhausted.

4.6 Optional Extension: Reviewer Agent

If time permits, an independent reviewer agent may inspect a proposed patch before final submission, focusing on whether the reported failure has been addressed, potential regressions, and relevant edge cases. This component is treated as an optional extension and is not part of the main contribution.

4.7 Comparisons and Ablations

We plan to compare four controlled conditions:

- **Baseline A:** original mini-SWE-agent;
- **Baseline B:** mini-SWE-agent with BM25-style lexical localization;

- **Method C:** mini-SWE-agent with lexical and structural file rankings fused using RRF;
- **Method D:** the same initial localization as Condition C, with additional reranking from usable runtime failure evidence.

The primary comparison is between A and D, while B and C help isolate the contribution of individual localization components. Additional ablations may be included if resources permit.

5. Preliminary Evaluation Plan

We will evaluate the original mini-SWE-agent (A), lexical localization (B), structure-aware localization (C), and structure plus failure-aware refinement (D) on the same held-out SWE-bench Verified tasks. The primary comparison is A versus D; B and C indicate which localization components contribute to any difference. We will assess repair outcomes, localization quality, and exploration cost separately so that a faster search is not mistaken for a successful repair.

5.1 Repair Performance

The primary outcome is *instance resolution rate*: the proportion of selected tasks marked **resolved** by the official SWE-bench evaluation harness^[4;10]. A run submits one final patch. The harness applies it in the task environment and checks that all **FAIL_TO_PASS** tests pass while the **PASS_TO_PASS** tests remain passing. A missing or unapplicable patch, or a run that exhausts its budget, counts as unresolved. We will also report patch-application rate and the fractions of the two test groups passed, to distinguish incomplete fixes from regressions. Tests run by the agent during repair provide feedback but do not replace the official final evaluation.

5.2 Localization Performance

For offline scoring, the files changed by the benchmark’s reference patch define the ground-truth locations. We will include changed Python source files and exclude test, documentation, and generated files. For each ranked list, *Hit@1* and *Hit@5* indicate whether at least one ground-truth file appears in the first one or five positions; mean reciprocal rank uses the rank of the first such file. We will also report file Recall@5 because some fixes require changes in multiple files. Tasks without a changed Python source file will be excluded from file-localization scoring, with the exclusion count reported.

Explicit ranking metrics will be reported for Conditions B–D; Condition A will be compared through repair outcomes and trajectory-based exploration measures.

Initial rankings will be scored before the agent executes any command. For D, we will additionally compare rankings before and after the first usable runtime failure, as defined in Section 3.6, reporting how many tasks actually produced such evidence. As a secondary analysis, changed lines will be mapped to functions in the task’s original Python AST; function Hit@5 will be reported only where this mapping is unambiguous, together with its task coverage. Reference patches and benchmark test labels will never be shown to the agent or used by its ranking method.

5.3 Exploration Efficiency

From saved action and observation trajectories, we will count unique Python source files whose contents the agent inspects and the number of model steps before it first inspects a ground-truth file. Runs that never reach such a file will be reported separately rather than assigned an artificial step count. We will also measure total input and output tokens and wall-clock time per task, including indexing and ranking overhead. Shell-command count and estimated API cost will be retained as supplementary measures. All added localization messages count toward the method’s token usage and cost.

5.4 Agent-Trajectory Analysis

We will inspect representative successes and failures using the same trajectory logs. A preliminary failure taxonomy assigns the main reason for each unsuccessful run to: (i) no relevant file inspected, (ii) relevant code inspected but the cause misdiagnosed, (iii) a patch that is missing, invalid, or incomplete, (iv) a regression or inadequate validation, or (v) an environment or resource-limit failure. Pilot runs will be used to refine coding rules before the held-out analysis. Case studies will show the candidate rankings, files inspected, test output, final patch, and official test result, including cases where localization improves but repair still fails.

5.5 Variability and Experimental Protocol

We provisionally plan approximately 20 development tasks and up to 100 disjoint held-out evaluation tasks from SWE-bench Verified, spanning multiple repositories. A pilot will establish the feasible task count and per-run budget; the final task list and protocol will be frozen before inspecting held-out results. All conditions will use identical task instances, the same pinned mini-SWE-agent and benchmark versions, the same LLM version and decoding settings, the same base prompts and execution environment, and equal step, token/cost, and wall-time limits. Only the prescribed localization context will differ. The final model configuration and resource limits will be recorded after the development pilot and fixed before held-out evaluation, as specified in Section 3.3.

The main study will submit one independent patch per task and condition. To assess run-to-run variability, we provisionally plan three independent runs in total per condition, including the main run, on an evaluation subset of up to 20 tasks chosen before results are examined. The final task count and repeat subset size will be determined from pilot costs and available API credit before held-out evaluation. Repeated runs will be reported individually and as averages, not combined into a best-of-three score. We will report paired task-level differences in resolution and efficiency, with confidence intervals where the final sample size permits. Infrastructure failures will be logged separately, with any exclusions disclosed.

6. Risks, Limitations, and Responsible Practice

- **Misleading localization.** Textual matches, structural relationships, and traceback locations may not identify the root cause. Candidate rankings may therefore misdirect exploration. They will serve as guidance rather than restrict access to other code.

- **Evaluation limitations.** Reference-patch locations are imperfect localization targets because alternative correct fixes may modify different code. Test-based evaluation also cannot establish complete correctness. Localization, repair outcomes, and representative trajectories will be interpreted together.
- **Benchmark contamination.** Public issues and solutions may have appeared in model training data. Separating development and evaluation tasks and withholding reference patches reduces experimental leakage, but cannot eliminate pretraining contamination.
- **Model variability and cost.** Repeated runs may produce different results, while additional localization can increase token usage and runtime. We will use the controlled settings, resource limits, and repeated-run analysis described in Section 5.
- **Environment failures.** Dependency or setup problems may prevent valid execution independently of repair quality. These failures and any resulting exclusions will be documented.
- **Limited generalizability.** Findings from a selected set of Python tasks and the chosen model configuration may not generalize to other languages, repositories, or models.

6.1 Safeguards

Experiments will run in task-specific containerized environments consistent with the SWE-bench infrastructure, as described in Section 3. Each environment will contain only the corresponding public repository, its required dependencies, and the available testing infrastructure. Credentials, confidential code, and unrelated host directories will be excluded from shared materials. Repository content and tool outputs will be treated as untrusted inputs. Generated patches will require human review before practical use.

References

- [1] Hongzheng Chai, Jiakun Li, Hongyue Yu, and Yuan Yuan. RepoNav: From Snippet Retrieval to File-Centered Repository Navigation for Code Agents, 2026.
- [2] Zhaoling Chen, Xiangru Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. LocAgent: Graph-Guided LLM Agents for Code Localization, 2025.
- [3] Zhonghao Jiang, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. Issue Localization via LLM-Driven Iterative Code Graph Searching, 2025.
- [4] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024.
- [5] Jiawei Liu, Yun Lin, Chenyan Liu, Yu Qian, Yiming Liu, Jiaxin Chang, Weinan Zhang, and Linpeng Huang. IssueExec: A Test-Driven Approach for Localizing Software Engineering Issues, 2026.

- [6] Wei Liu, Chao Peng, Pengfei Gao, Aofan Liu, Wei Zhang, Haiyan Zhao, and Zhi Jin. GraphLocator: Graph-guided Causal Reasoning for Issue Localization, 2025.
- [7] mini-SWE-agent contributors. mini-SWE-agent. Software repository, n.d. Accessed 2026-09-22.
- [8] mini-SWE-agent contributors. mini-SWE-agent v2.0 Migration Guide. Official documentation, n.d. Accessed 2026-09-22.
- [9] OpenAI. Introducing SWE-bench Verified. Official benchmark announcement, 2024. Accessed 2026-09-22.
- [10] SWE-bench contributors. SWE-bench evaluation harness reference. Official documentation. Accessed 2026-09-25.
- [11] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based Software Engineering Agents, 2024.
- [12] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering, 2024.
- [13] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous Program Improvement, 2024.
- [14] Jian Zhou, Hongyu Zhang, and David Lo. Where should the bugs be fixed? more accurate information retrieval-based bug localization based on bug reports. In *Proceedings of the 34th International Conference on Software Engineering*, pages 14–24, 2012.